

Chapter4. word2vec 속도 개선

🌟목표 : word2vec가 느린 원인과 원인에 따른 해결방안 찾기!!🌟

말뭉치의 어휘 수가 많아지면 계산량이 커지는 단점이 있다.

why?

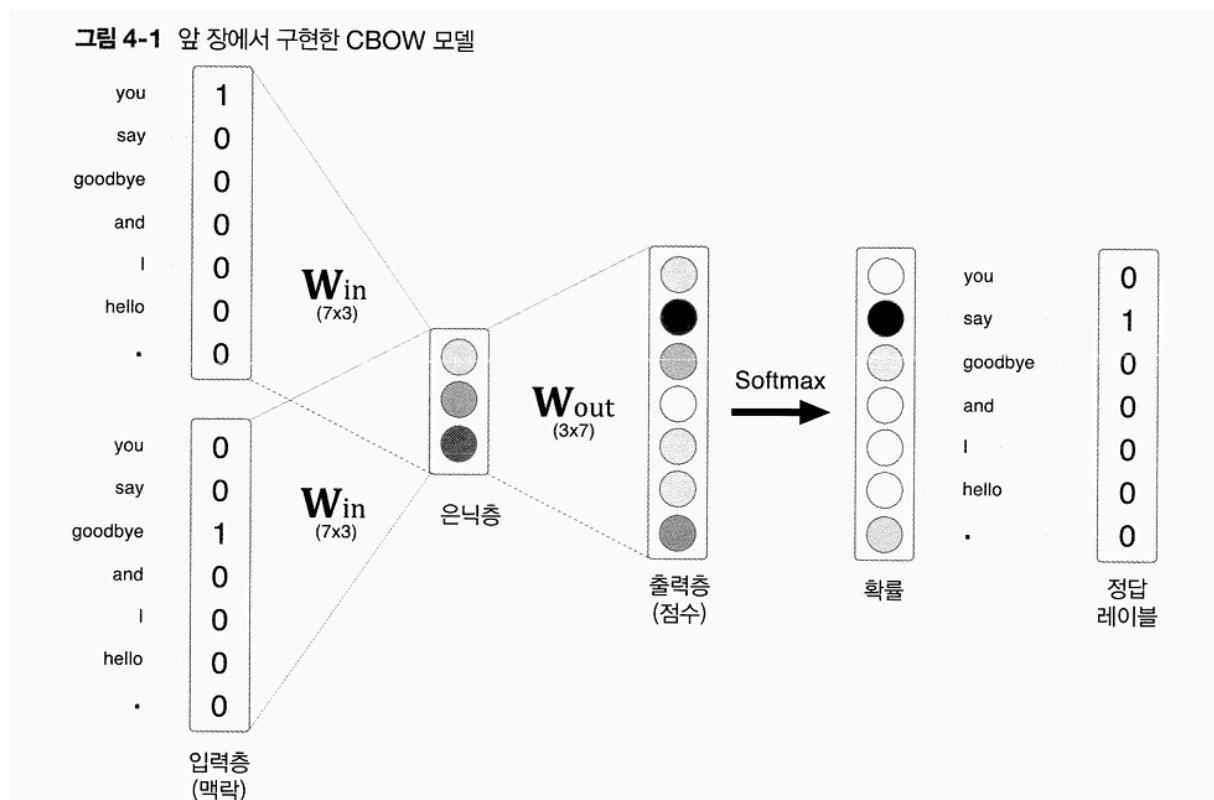
원인1. 원핫 포현과 가중치 행렬의 곱 계산

원인2. 은닉층과 가중치 행렬의 곱 및 softmax 계층 계산

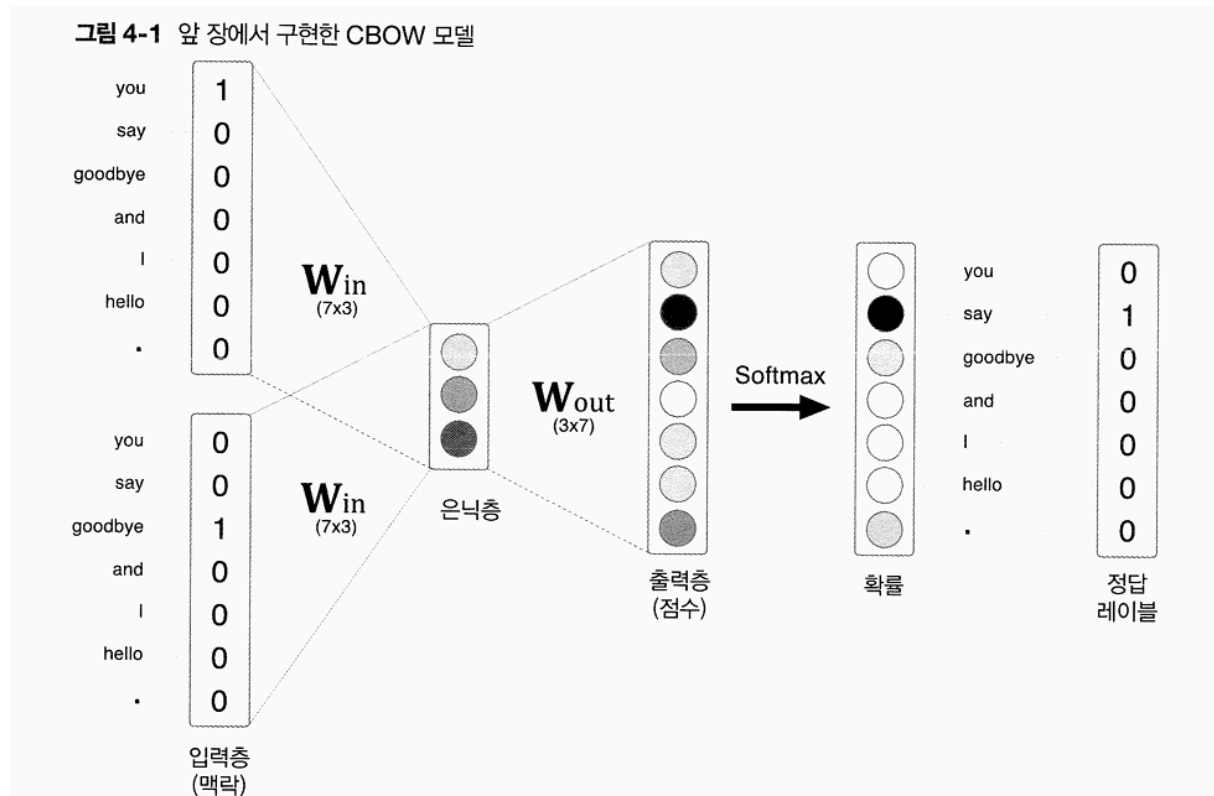
이를 해결하고자 한다

4.1. word2vec 개선

원인1. 원핫 포현과 가중치 행렬의 곱 계산 해결



이때 다루는 어휘의 수, 말뭉치가 100만개라면



뉴런도 100만개가 존재할 것

→ 수많은 뉴런 때문에 중간 계산에서 많은 시간 소요

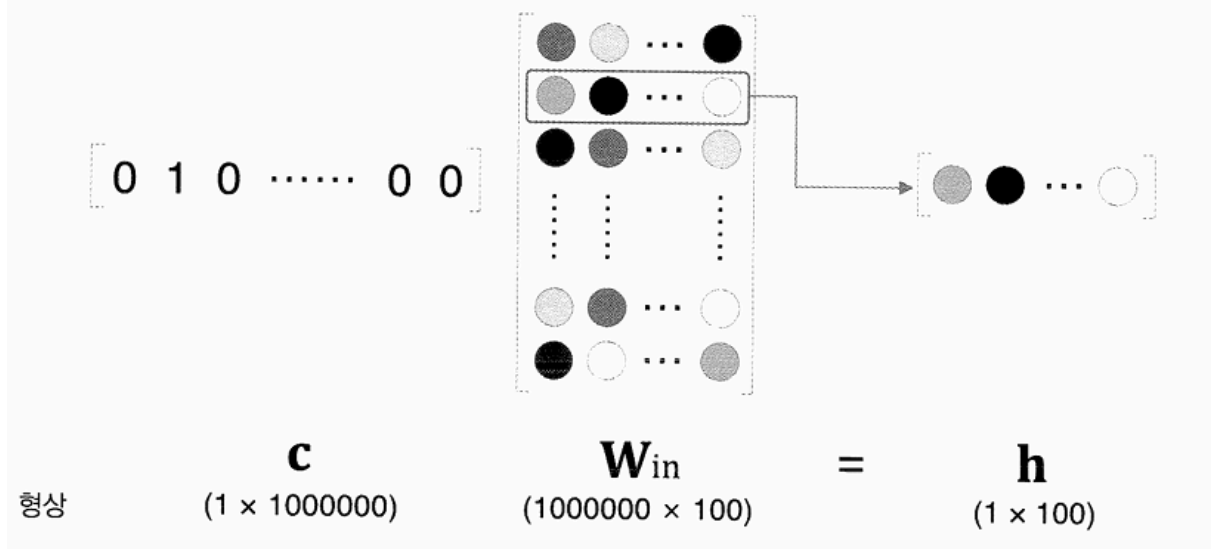
원인1. 원핫 포현과 가중치 행렬의 곱 계산

원인2. 은닉층과 가중치 행렬의 곱 및 softmax 계층 계산

4.1.1. Embedding 계층

어휘의 수가 100만개라면,

그림 4-3 맥락(원핫 표현)과 MatMul 계층의 가중치를 곱한다.



말뭉치가 100만개의 어휘를 담고있다면, 단어의 원핫 표현도 100만차원이 된다.

하지만 결과적으로는 특정 행을 추출하는 것이 목표

So, MatMul 계층의 행렬 곱 계산이 필요가 없음

→ Embedding 계층 구현

```
import numpy as np
```

```
W = np.arange(21).reshape(7,3)
```

```
W  
  
array([[ 0,  1,  2],  
       [ 3,  4,  5],  
       [ 6,  7,  8],  
       [ 9, 10, 11],  
       [12, 13, 14],  
       [15, 16, 17],  
       [18, 19, 20]])
```

```
W[2]  
  
array([6, 7, 8])
```

```
W[5]  
  
array([15, 16, 17])
```

→ backward 까지 구현

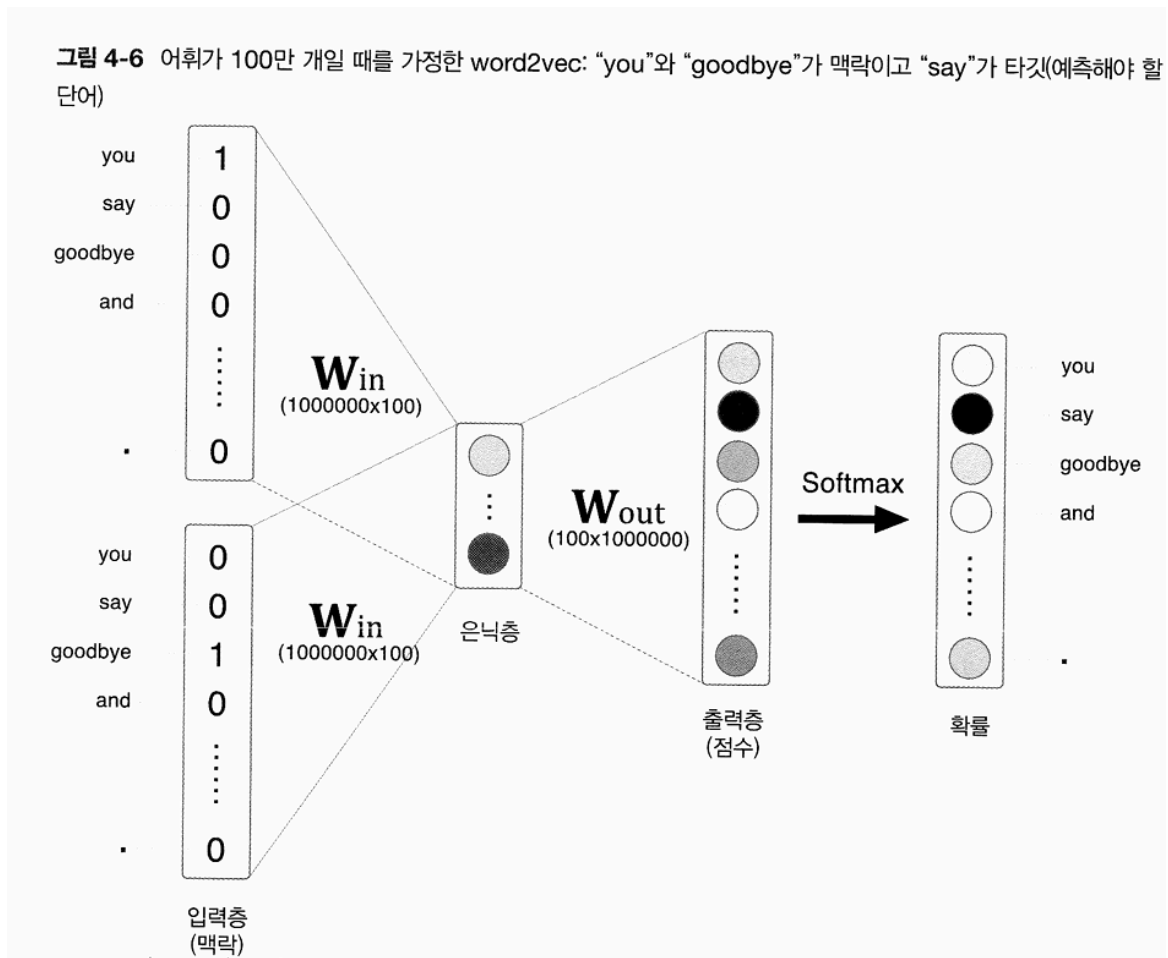
```
class Embedding:  
    def __init__(self, W):  
        self.params = [W]  
        self.grads = [np.zeros_like(W)]  
        self.idx = None  
  
    def forward(self, idx):  
        self.idx = idx  
        out = W[idx]  
        return out  
  
    def backward(self, dout):  
        dW, = self.grads  
        dW[...] = 0  
  
        for i, word_id in enumerate(self.idx):  
            dW[word_id] += dout[i]  
  
        return None
```

4.2. word2vec 개선

원인2. 은닉층과 가중치 행렬의 곱 및 softmax 계층 계산

네거티브 샘플링기법을 활용하여 계산량을 억제하겠다

4.2.1. 은닉층 이후 계산의 문제점



goodbye & you가 맥락이고 say가 target이라면, 위와같이 확률을 뽑아내야 한다

은닉층 이후에서 계산이 오래걸리는 부분은 총 2가지

1. 은닉층의 뉴런과 가중치 행렬곱
2. softmax 계층 계산

$$y_k = \frac{\exp(s_k)}{\sum_{i=1}^{1000000} \exp(s_i)}$$

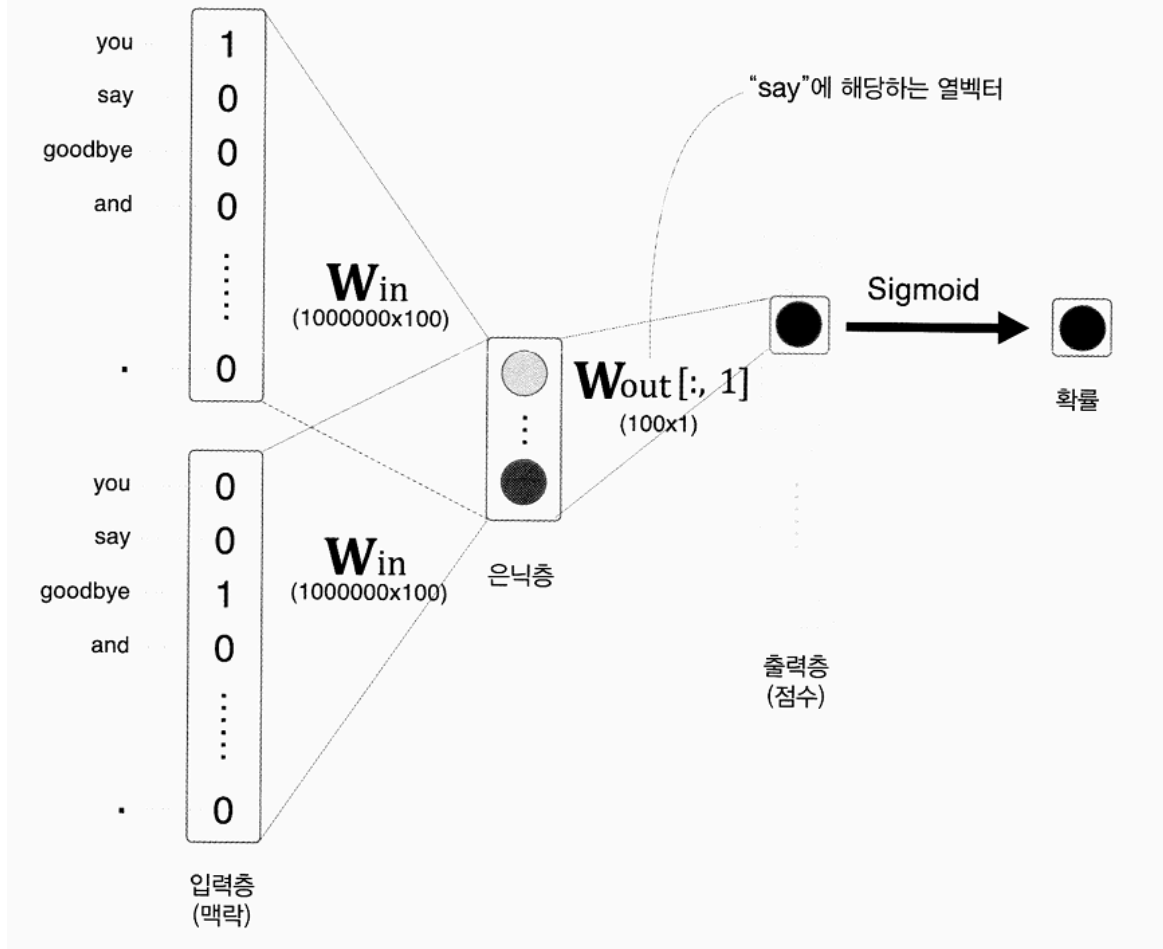
- k번째 원소(단어)를 타겟으로 했을 때 Softmax 계산식
- 분모를 얻기 위해선 exp계산을 100만번 해야한다.
- Softmax를 대신할 "가벼운"계산이 필요함

4.2.2. 다중 분류 → 이진 분류

네거티브 샘플링 기법이란?

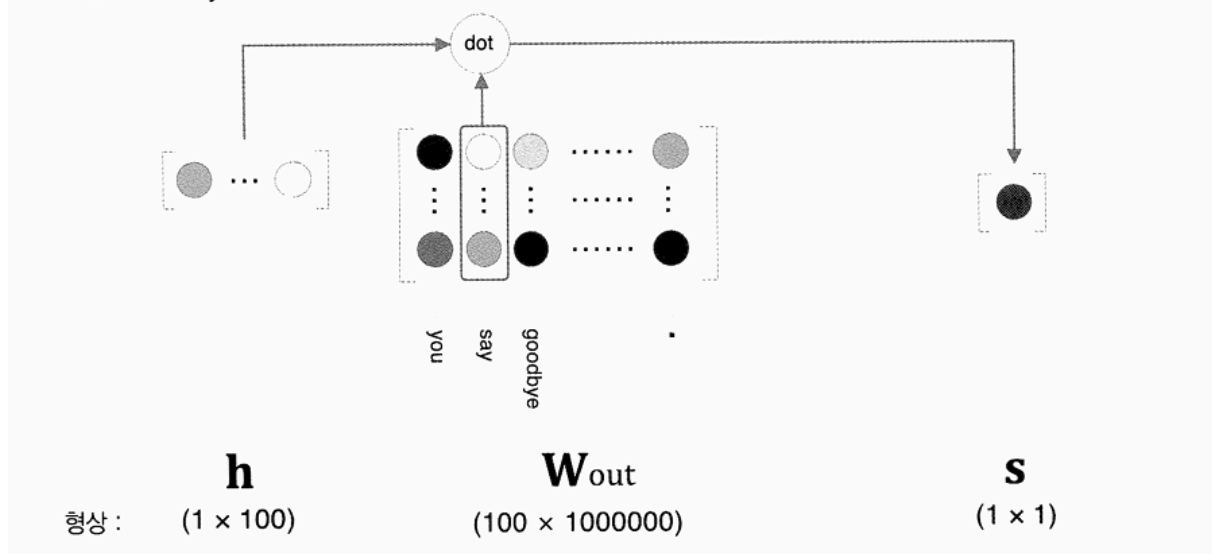
다중분류를 이진 분류로 근사하는 것

그림 4-7 타깃 단어만의 점수를 구하는 신경망



출력층의 뉴런이 하나이므로, 은닉층과 출력쪽 가중치 행렬의 내적은 say에 해당하는 열만 추출하고, 추출된 벡터와 은닉층 뉴런과 내적을 계산 → 이 내적 값이 최종 점수

그림 4-8 “say”에 해당하는 열벡터와 은닉층 뉴런의 내적을 계산한다('dot' 노드가 내적을 계산함).



4.2.3. 시그모이드 함수와 교차 엔트로피 오차

손실을 구하기 위해 손실 함수중 “교차 엔트로피 오차”를 사용한다.

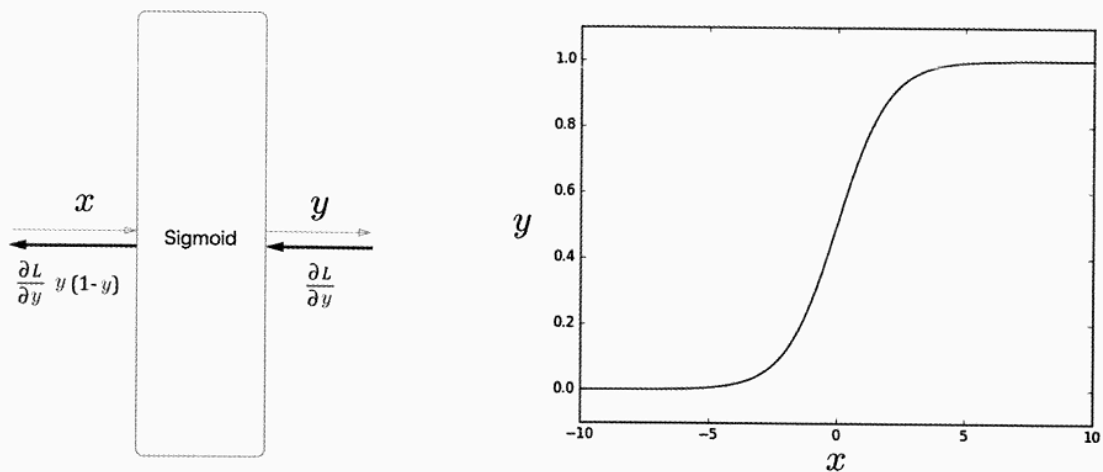
다중 분류는 출력층에서는 “Softmax function”

손실 함수로는 “교차 엔트로피 오차”

이진 분류는 출력층에서는 “시그모이드 함수”

$$y = \frac{1}{1 + \exp(-x)}$$

그림 4-9 Sigmoid 계층(왼쪽)과 시그모이드 함수의 그래프(오른쪽)



손실 함수로는 “교차 엔트로피 오차”

교차 엔트로피 오차

→ 시그모이드 함수를 이용해 확률 y 를 구하면

→ y : 시그모이드 함수의 출력

→ t : 정답 레이블 (1이면 정답이 yes, 0이면 정답이 no)

$$L = -(t \log y + (1 - t) \log(1 - y))$$

이 식을 이용하여 손실을 구한다

4.2.4. 다중 분류에서 이진 분류로 구현

그림 4-11 다중 분류를 수행하는 CBOW 모델의 전체 그림(Embedding 계층은 Embed로 표기)

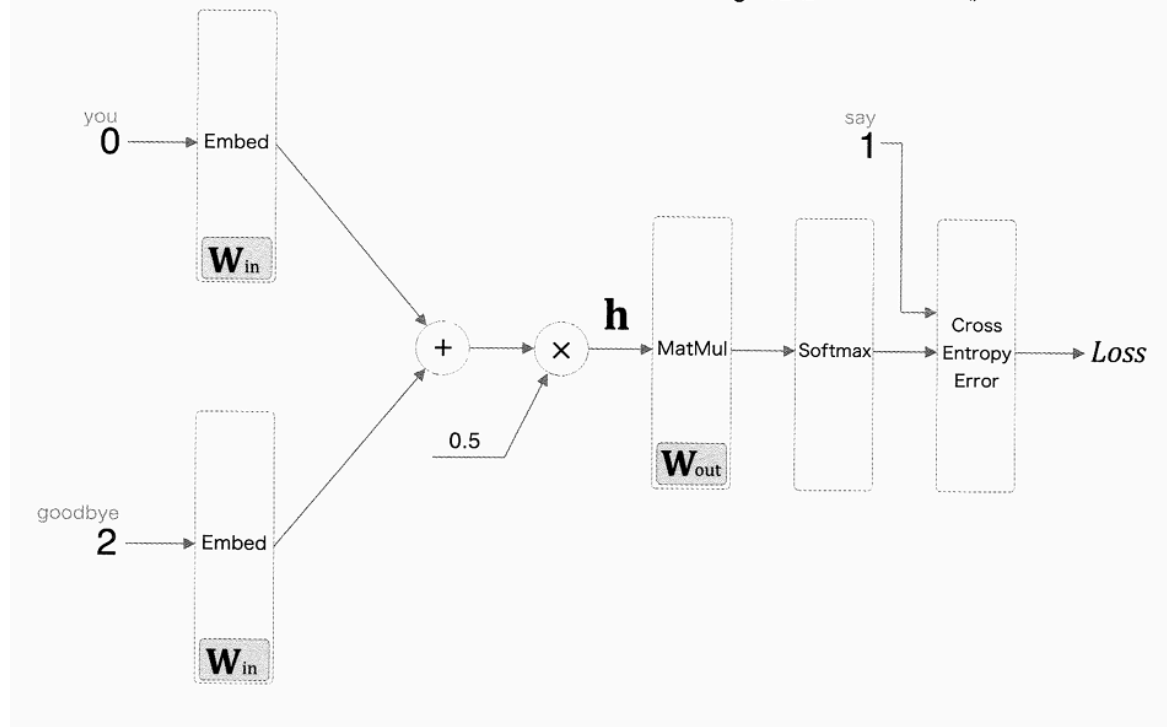
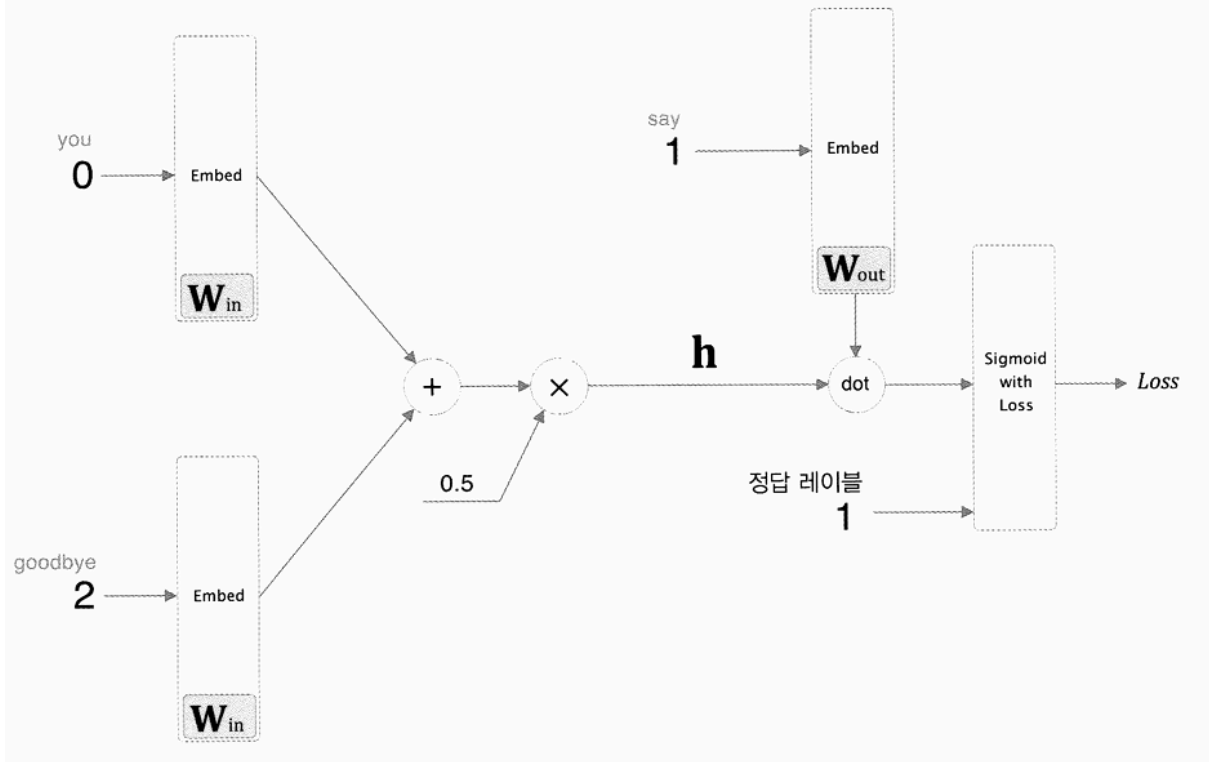


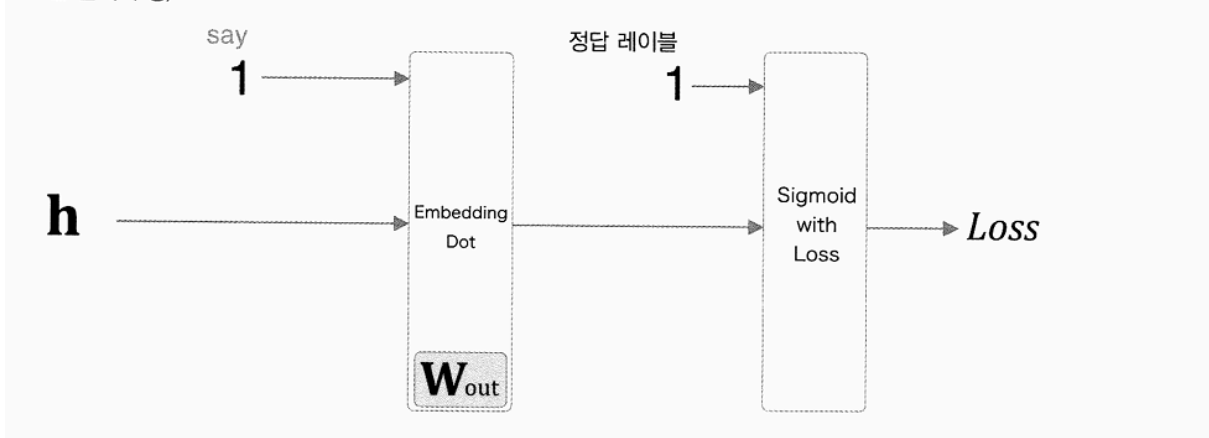
그림 4-12 이진 분류를 수행하는 word2vec(CBOW 모델)의 전체 그림



Embedding Dot 계층

: Embedding 계층 + dot(내적) 연산처리 계층

그림 4-13 [그림 4-12]의 은닉층 이후 처리(Embedding Dot 계층을 사용하여 Embedding 계층과 내적 계산을 한 번에 수행)



```

from typing_extensions import dataclass_transform
class EmbeddingDot :
    def __init__(self, W):
        self.embed = Embedding(W)
        self.params = self.embed.params
        self.grads = self.embed.grads
        self.cache = None

    def forward(self, h, idx):
        target_W = self.embed.forward(idx)
        out = np.sum(target_W * h, axis=1)

        self.cache = (h, target_W)
        return out

    def backward(self, dout):
        h, target_W = self.cache
        dout = dout.reshape(dout.shape[0], 1)

        dtarget_W = dout * h
        self.embed.backward(dtarget_W)
        dh = dout * target_W
        return dh

```

4.2.5. 네거티브 샘플링

그림 4-15 CBOW 모델의 은닉층 이후의 처리 예: 맥락은 “you”와 “goodbye”이고, 타깃이 “say”일 확률은 0.993(99.3%)이다.

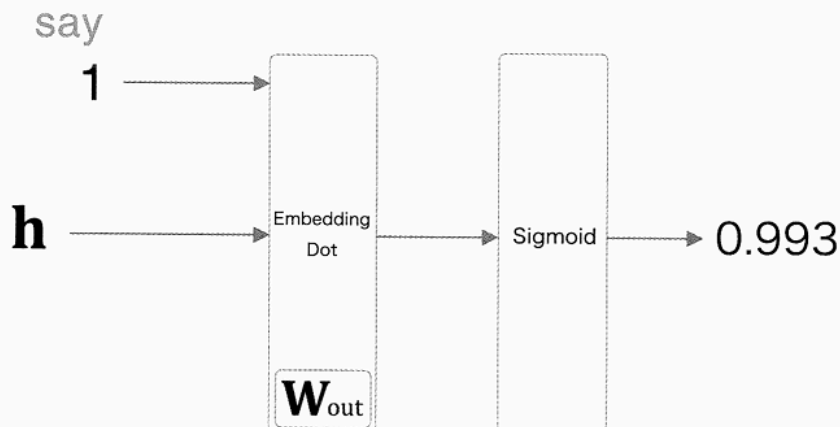
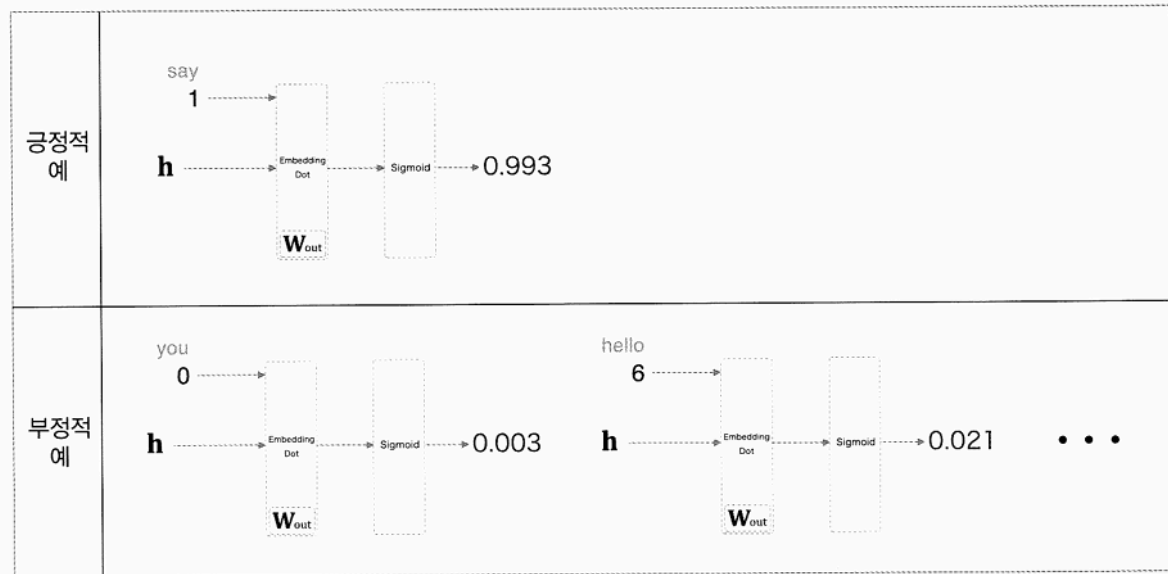


그림 4-16 긍정적 예(정답)를 “say”라고 가정하면, “say”를 입력했을 때의 Sigmoid 계층 출력은 1에 가깝고, “say” 이외의 단어를 입력했을 때의 출력은 0에 가까워야 한다. 이런 결과를 내어주는 가중치가 필요하다.



긍정적인 예시인 say에 대해서는 출력을 1에 가깝게 만들고,
부정적인 예시인 say 이외 단어에 대해서는 0에 가깝게 만들어야 한다.

그럼 부정적 예를 이용하여 학습시키면 어떻게 될까?

→ 안 된다

⇒ 모든 부정적인 예를 대상으로하면 어휘 양이 너무 많아지기 때문

⇒ So, 근사적인 해법으로 갯수를 선택하여 사용한다.

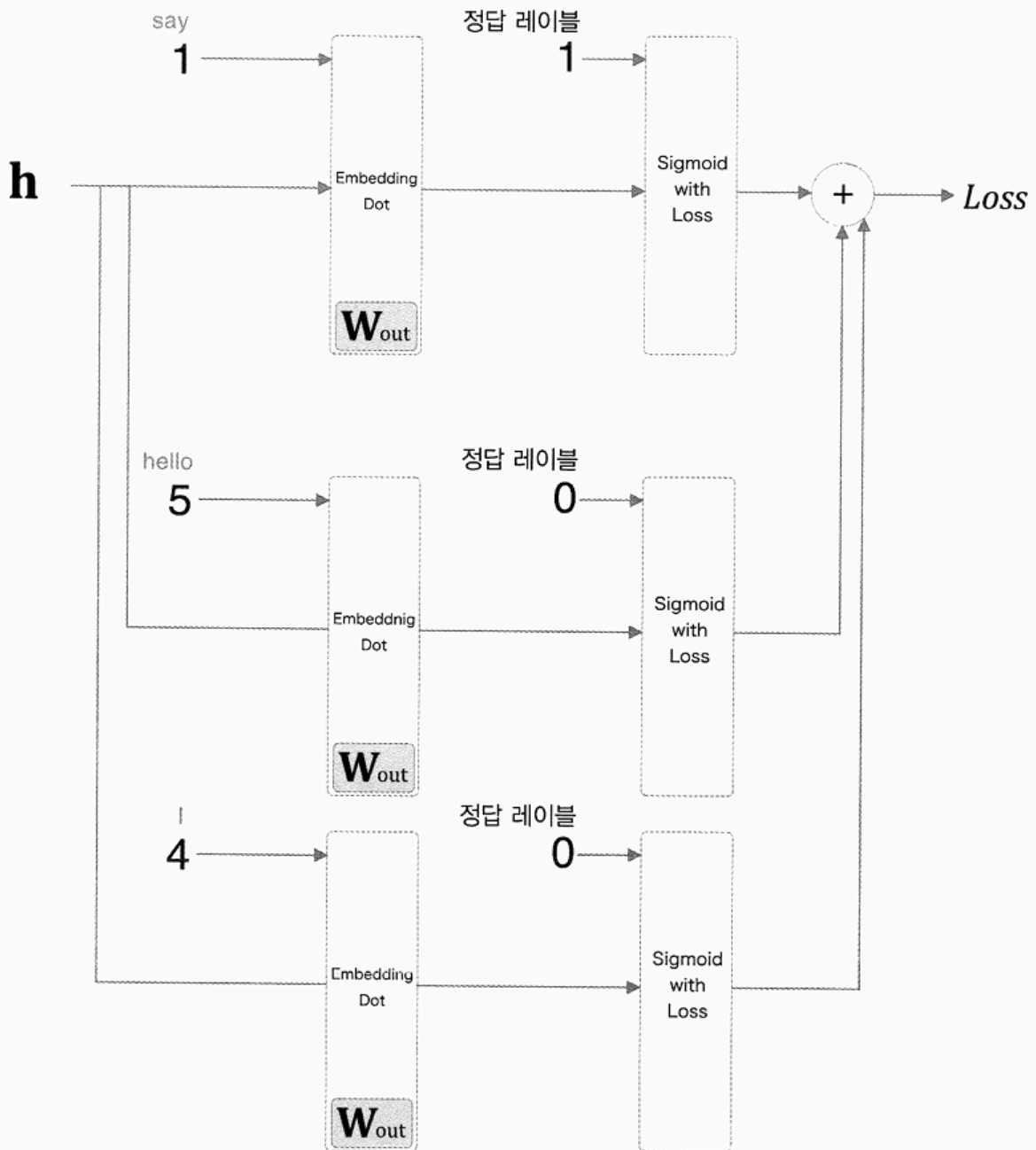
네거티브 샘플링**

→ 긍정적 예를 타겟으로 한 경우의 손실

→ 그와 동시에 부정적 예를 몇개 샘플링하여(선별) 부정적인 예에 대해서도 손실을 구함

그래서 손실을 더한 값을 최종 손실로 한다.

그림 4-17 네거티브 샘플링의 예(은닉층 이후의 처리에 주목하여 그린 계산 그래프)



4.2.6. 네거티브 샘플링의 샘플링 기법

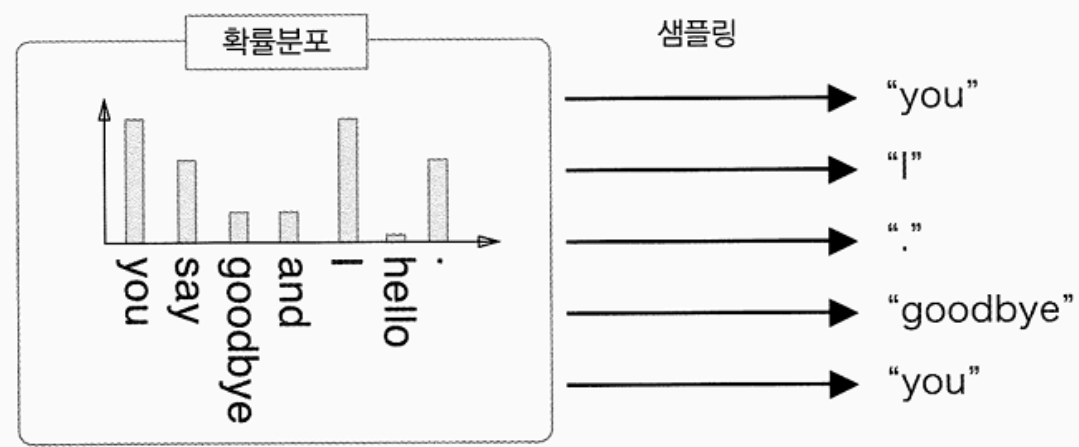
부정적인 것에 대한 샘플링

→ 드물게 등장하는 단어를 적게 추출하는 것으로,

출현 횟수를 구해서 확률 분포로 나타낸다

그리고 확률 분포대로 단어를 샘플링 하면 된다.

그림 4-18 확률분포에 따라 샘플링을 여러 번 수행한다.



코드에서 무작위로 샘플링

```
import numpy as np
```

```
np.random.choice(10)
```

8

```
np.random.choice(10)
```

6

```
words = ['you', 'say', 'goodbye', 'I', 'hello', '.']  
np.random.choice(words)
```

```
np.str_('goodbye')
```

```
#중복 허용 5개 무작위로 뽑기  
np.random.choice(words, size = 5)
```

```
array(['hello', 'goodbye', '.', 'goodbye', 'you'], dtype='<U7')
```

```
#중복 없이 5개 무작위로 뽑기  
np.random.choice(words, size = 5, replace = False)
```

```
array(['hello', 'I', 'you', 'say', 'goodbye'], dtype='<U7')
```

```
p = [0.5, 0.1, 0.05, 0.2, 0.05, 0.1]  
np.random.choice(words, p = p)
```

```
np.str_('hello')
```

$$P'(w_i) = \frac{P(w_i)^{0.75}}{\sum_j^n P(w_j)^{0.75}}$$

권고하는 네거티브 샘플링 식

4.2.7. 네거티브 샘플링 구현

초기화

```
class NegativeSamplingLoss:
    def __init__(self, W, corpus, power=0.75, sample_size=5):
        self.sample_size = sample_size
        self.sampler = UnigramSampler(corpus, power, sample_size)
        self.loss_layers = [SigmoidWithLoss() for _ in range(sample_size + 1)]
        self.embed_dot_layers = [EmbeddingDot(W) for _ in range(sample_size + 1)]

        self.params, self.grads = [], []
        for layer in self.embed_dot_layers:
            self.params += layer.params
            self.grads += layer.grads
```

Plus) 긍정적 예 + 부정적 예

```
def forward(self, h, target):
    batch_size = target.shape[0]
    negative_sample = self.sampler.get_negative_Sample(target)

    #긍정적 예
    score = self.embed_dot_layers[0].forward(h, target)
    correct_label = np.ones(batch_size, dtype=np.int32)
    loss = self.loss_layers[0].forward(score, correct_label)

    #부정적 예
    negative_label = np.zeros(batch_size, dtype=np.int32)
    for i in range(self.sample_size):
        negative_target = negative_sample[:, i]
        score = self.embed_dot_layers[1 + i].forward(h, negative_target)
        loss += self.loss_layers[1+i].forward(score, negative_label)

    return loss
```



```

class CBOW:
    def __init__(self, vocab_size, hidden_size, window_size, corpus):
        V,H = vocab_size, hidden_size

        W_in = 0.01*np.random.randn(V,H),astype('f')
        W_out = 0.01*np.random.randn(V,H),astype('f')

        #계층
        self.in_layers = []
        for i in range(2*window_size):
            layer = Embedding(W_in)
            self.in_layers.append(layer)
        self.ns_loss = NegativeSamplingLoss(W_out, corpus, power=0.75, sample_size=5)

        layers = self.in_layers + [self.ns_loss]
        self.params, self.grads = [], []
        for layer in layers:
            self.params += layer.params
            self.grads += layer.grads

        self.word_vecs = W_in

```

그림 4-19 맥락과 타겟을 단어 ID로 나타낸 예(맥락의 윈도우 크기는 1)

맥락	타겟		맥락	타겟
you, goodbye	say	단어 ID →	[[0 2]	[1
say, and	goodbye		[1 3]	2
goodbye, I	and		[2 4]	3
and, say	I		[3 1]	4
I, hello	say		[4 5]	1
say, .	hello		[1 6]]	5]

```

pkl_file = 'cbow_params.pkl'

with open(pkl_file, 'rb') as f:
    params = pickle.load(f)
    word_vecs = params['word_vecs']
    word_to_id = params['word_to_id']
    id_to_word = params['id_to_word']

querys = ['you', 'year', 'car', 'toyota']
for query in querys:
    most_similar(query, word_to_id, id_to_word, word_vecs, top=5)

```

```

[query] you
we: 0.610352
someone: 0.590820
i: 0.554199
something: 0.489990
anyone: 0.473145

```

```

[query] year
month: 0.718262
week: 0.652344
spring: 0.627441
summer: 0.625977
decade: 0.603516

```

```

[query] car
luxury: 0.497314
arabia: 0.478027
auto: 0.471436
disk-drive: 0.450684
travel: 0.408936

```

```

[query] toyota
ford: 0.550781
instrumentation: 0.510254
mazda: 0.493896
bethlehem: 0.475342
nissan: 0.474854

```

이렇게 벡터를 구현하면,

woman - man = ? - king

이때 ? 를 woman으로 추론할 수 있게 된다

4.4. word2vec 남은 주제

4.4.1. word2vec을 사용한 어플리케이션 예시

비슷한 단어를 찾는 용도 뿐만 아니라

전이학습이 핵심이다

텍스트 분류, 문서 클러스터링, 품사 태그 달기, 감정 분석 등..

자연어 처리 작업이면 가장 먼저 단어를 벡터로 변환하는 작업을 해야하고,

학습이 완료된 단어의 분산 표현을 이용 가능

문장의 각 단어를 분산 표현으로 변환하여 합을 구하는 것

→ **bag-of-words**

: 단어의 순서를 고려하지 않는 모델

그림 4-21 단어의 분산 표현을 이용한 시스템의 처리 흐름



구체적인 예시 :

수많은 메일을 자동으로 분류하는 시스템

감정을 3단계로 구분하려면?

1st 데이터 (메일) 수집

사용자가 보낸 메일을 모으고, 모은 메일들에 수동으로 레이블 붙이기

3단계 감정이니, 긍정/중립/부정 3가지를 붙인다.

2nd 학습된 word2vec 이용

메일을 벡터로 변환한다.

3rd 학습 수행

감정 분석을 수행하는 분류 시스템 (ex] SVM이나 신경망 등)에 레이블을 입력하여 학습 수행

4.4.2. 단어 벡터 평가 방법

word2vec을 통해 단어의 분산 표현을 얻을 수 있다.

분산 표현이 좋은지 판단은 어떻게 할까?

보통 실제 애플리케이션과 분리해 평가한다.

단어 유사도 검증 세트를 사용해 평가하게 되는데,

cat -8점- animal

cat -2점- car

⇒ 이런식으로 사람이 단어 사이의 유사 정도 규정

그림 4-23 유추 문제에 의한 단어 벡터의 평가 결과(논문 [27]에서 발췌)

모델	차수	말뭉치 크기	의미(semantic)	구문(syntax)	종합
CBOW	300	16억	16.1	52.6	36.1
skip-gram	300	10억	61	61	61
CBOW	300	60억	63.6	67.4	65.7
skip-gram	300	60억	73.0	66.0	69.1
CBOW	1000	60억	57.3	68.9	63.7
skip-gram	1000	60억	66.1	65.1	65.6

1. 모델에 따라 정확도가 다르다

→ 말뭉치에 따라 적합한 모델이 선택되어야한다

2. 일반적으로 말뭉치가 클수록 결과가 좋음

→ 항상 data가 많은게 좋아

3. 단어 벡터의 차원 수는 적당한 크기가 좋다

→ 너무 커도 정확도가 나빠진다

4.5. 정리

핵심은 **일부**를 처리한다는 것

모든 데이터를 처리하는 것이 비현실적이며, 필요한 일부에 집중해야함

⇒ 네거티브 샘플링

이번 장에서 배운 내용

- Embedding 계층은 단어의 분산 표현을 담고 있으며, 순전파 시 지정한 단어 ID의 벡터를 추출한다.
- word2vec은 어휘 수의 증가에 비례하여 계산량도 증가하므로, 근사치로 계산하는 빠른 기법을 사용하면 좋다.
- 네거티브 샘플링은 부정적 예를 몇 개 샘플링하는 기법으로, 이를 이용하면 다중 분류를 이진 분류처럼 취급할 수 있다.
- word2vec으로 얻은 단어의 분산 표현에는 단어의 의미가 녹아들어 있으며, 비슷한 맥락에서 사용되는 단어는 단어 벡터 공간에서 가까이 위치한다.
- word2vec의 단어의 분산 표현을 이용하면 유추 문제를 벡터의 덧셈과 뺄셈으로 풀 수 있게 된다.
- word2vec은 전이 학습 측면에서 특히 중요하며, 그 단어의 분산 표현은 다양한 자연어 처리 작업에 이용할 수 있다.